

7

Ethereum 101

This chapter is intended to be an introduction to the Ethereum blockchain. You will be introduced to the fundamentals and advanced theoretical concepts behind Ethereum. A discussion on various components, protocols, and algorithms relevant to the Ethereum blockchain will be given in detail so that you can understand the theory behind this blockchain paradigm. Also, a practical and in-depth introduction to wallet software, mining, and setting up Ethereum nodes will be covered in this chapter. Some material on various challenges, such as security and scalability faced by Ethereum, will also be introduced. Additionally, trading and market dynamics will be discussed.

Introduction

Ethereum was conceptualized by *Vitalik Buterin* in November 2013. The key idea proposed was the development of a Turing-complete language that allows the development of arbitrary programs (smart contracts) for blockchain and decentralized applications. This is in contrast to bitcoin, where the scripting language is very limited and allows basic and necessary operations only.

Ethereum clients and releases

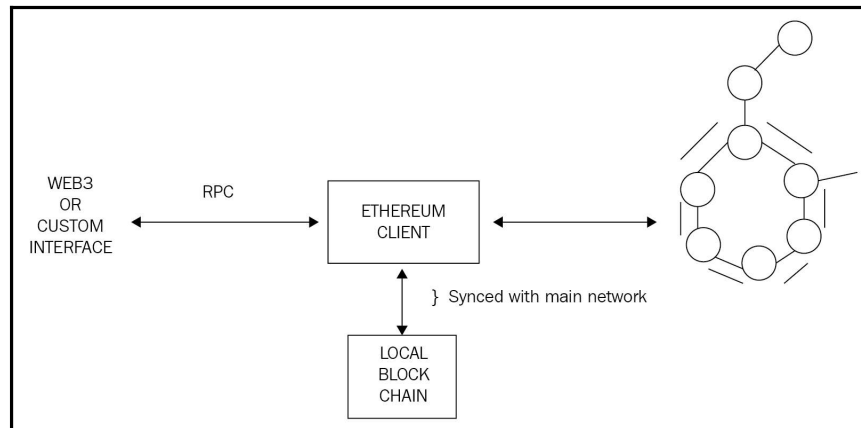
Various Ethereum clients have been developed using different languages and currently most popular are go-Ethereum and parity. go-Ethereum was developed using Golang, whereas parity was built using Rust. There are other clients available too, but usually, the go-Ethereum client known as *geth* is sufficient for all purposes. Mist is a user-friendly **Graphical User Interface (GUI)** wallet that runs *geth* in the background to sync with the network. More details on this will be provided later in the chapter, in the installation and mining section.

The first release of Ethereum was known as *Frontier*, and the current release of Ethereum is called *homestead release*. The next version is named metropolis and it focuses on protocol simplification and performance improvement. The final release is named *serenity*, which is envisaged to have a Proof of Stake algorithm (Casper) implemented with it. Other areas of research targeted with serenity include scalability, privacy, and **Ethereum virtual machine (EVM)** upgrade. As this is a continuous development effort and the Ethereum ecosystem will undergo constant improvement and development, serenity should not really be considered a *final* version but a major milestone in a long journey of continuous improvement. Further releases are envisaged but have not been named yet. The vision of *web 3.0* has already been proposed and is being discussed in the community. Web 3.0 is a concept that basically proposes a semantic and intelligent web as an evolution of the existing web 2.0 technology. This is the vision of an ecosystem where people, applications, data, and web are all connected together and are able to interact with each other in an intelligent fashion. With the advent of the blockchain technology, an idea of decentralized web has also emerged, which in fact was the original vision of the Internet. The core idea is that all major services, such as DNS, search engines, and identity on the Internet will be decentralized in web 3.0. This is where Ethereum is being envisaged as a platform that can help realize this vision.

The Ethereum stack

The Ethereum stack consists of various components. At the core, there is the Ethereum blockchain running on the P2P Ethereum network. Secondly, there's an Ethereum client (usually geth) that runs on the nodes and connects to the peer-to-peer Ethereum network from where blockchain is downloaded and stored locally. It provides various functions, such as mining and account management. The local copy of the blockchain is synchronized regularly with the network. Another component is the `web3.js` library that allows interaction with geth via the **Remote Procedure Call (RPC)** interface.

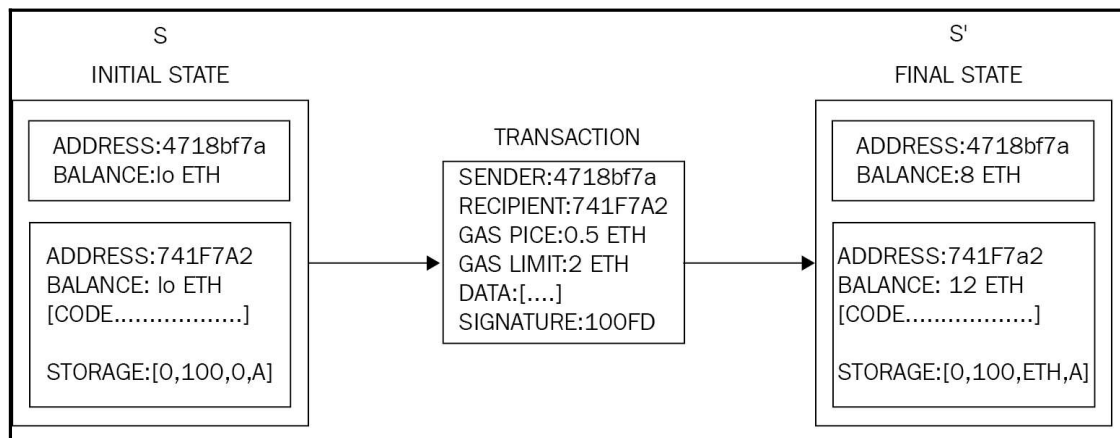
This can be visualized in the following diagram:



The Ethereum stack showing various components

Ethereum blockchain

Ethereum, just like any other blockchain, can be visualized as a transaction-based state machine. This is mentioned in the Ethereum yellow paper written by *Dr. Gavin Wood*. The idea is that a genesis state is transformed into a final state by executing transactions incrementally. The final transformation is then accepted as the absolute undisputed version of the state. In the following diagram, the Ethereum state transition function is shown, where a transaction execution has resulted in a state transition.



Ethereum State transition function

In the preceding example, a transfer of 2 Ether from **Address 4718bf7a** to **Address 741f7a2** is initiated. The initial state represents the state before the transaction execution and the final state is what the morphed state looks like. This will be discussed in more detail later in the chapter, but the aim of this example is to introduce the core idea of state transition in Ethereum.

Currency (ETH and ETC)

As an incentive to the miners, Ethereum also rewards its native currency called Ether, abbreviated as ETH. After the DAO hack (described later), a hard fork was proposed in order to mitigate the issue; therefore, there are now two Ethereum blockchains: one is called Ethereum classic and its currency is represented by ETC, whereas the hard-forked version is ETH, which continues to grow and on which active development is being carried out. ETC, however, has its own following with a dedicated community that is further developing ETC, which is the nonforked original version of Ethereum. This chapter is focused mainly on ETH, which is the currently the most active and official Ethereum blockchain.

Forks

With the latest release of homestead, due to major protocol upgrades, it resulted in a hard fork. The protocol was upgraded at block number 1,150,000, resulting in the migration from the first version of Ethereum known as Frontier to the second version of Ethereum called homestead.

A recent unintentional fork that occurred on November 24, 2016, at 14:12:07 UTC was due to a bug in the geth client's journaling mechanism. Network fork occurred at block number 2,686,351. This bug resulted in geth failing to revert empty account deletions in the case of the empty out-of-gas exception. This was not an issue in parity (another popular Ethereum client). This means that from block number 2686351, the Ethereum blockchain is split into two, one running with parity clients and the other with geth. This issue was resolved with the release of geth version 1.5.3.

Gas

Another key concept in Ethereum is that of gas. All transactions on the Ethereum blockchain are required to cover the cost of computation they are performing. The cost is covered by something called *gas* or *crypto fuel*, which is a new concept introduced by Ethereum. This gas as *execution fee* is paid upfront by the transaction originators. The *fuel* is consumed with each operation. Each operation has a predefined amount of gas associated with it. Each transaction specifies the amount of gas it is willing to consume for its execution. If it runs *out of gas* before the execution is completed, any operation performed by the transaction up to that point is rolled back. If the transaction is successfully executed, then any remaining gas is refunded to the transaction originator.

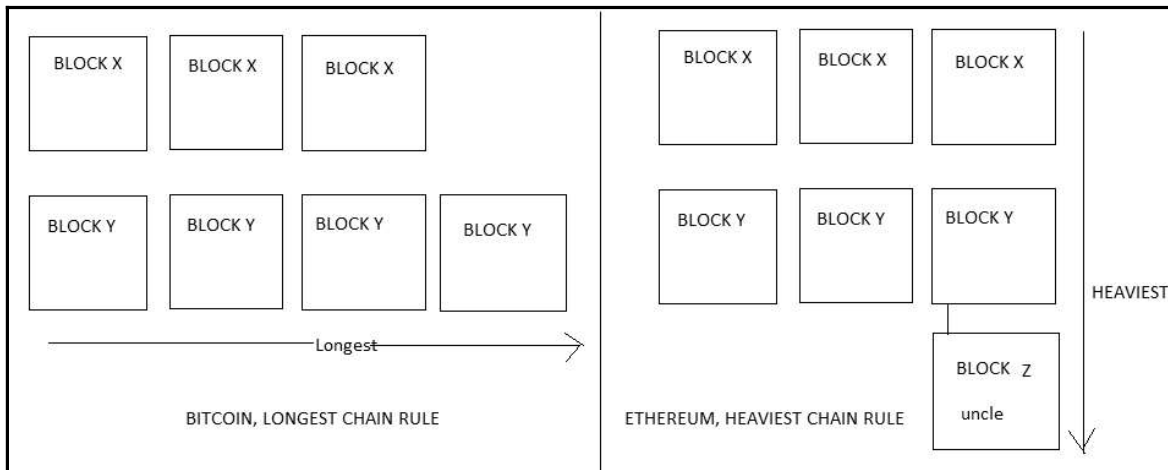
This concept should not be confused with mining fee, which is a different concept that is used to pay *gas* as a fee to the miners. More information on the concept and calculations related to gas and operations will be provided later in the chapter.

The consensus mechanism

The consensus mechanism in Ethereum is based on the GHOST protocol originally proposed by Zohar and Sompolinsky in December 2013. Those of you interested in it can explore the detailed original paper at <http://eprint.iacr.org/2013/881.pdf>.

Ethereum uses a simpler version of this protocol, where the chain that has most computational effort spent on it in order to build it is identified as the definite version. Another way of looking at it is to find the longest chain, as the longest chain must have been built by consuming adequate mining effort. **Greedy Heaviest Observed Subtree (GHOST)** was first introduced as a mechanism to alleviate the issues arising out of fast block generation times that led to stale or orphan blocks. In GHOST, stale blocks are added in calculations to figure out the longest and heaviest chain of blocks. Stale blocks are called Uncles or Ommers in Ethereum.

The following diagram shows a quick comparison between the longest and heaviest chain:



Longest versus heaviest chain

The world state

The world state in Ethereum represents the global state of the Ethereum blockchain. It is basically a mapping between Ethereum addresses and account states. The addresses are 20 bytes long. This mapping is a data structure that is serialized using **Recursive Length Prefix (RLP)**. RLP is a specially developed encoding scheme that is used in Ethereum to serialize binary data for storage or transmission over the network and also to save the state in a Patricia tree. The RLP function takes an item as an input, which can be a string or a list of items, and produces raw bytes that are suitable for storage and transmission over the network. RLP does not encode data; instead, its main purpose is to encode structures.

The account state

The account state consists of four fields: nonce, balance, storageroot and codehash and is described in detail here.

Nonce

This is a value that is incremented every time a transaction is sent from the address. In case of contract accounts, it represents the number of contracts created by the account. Contract accounts are one of the two types of accounts that exist in Ethereum; they will be explained later on in the chapter in more detail.

Balance

This value represents the number of Weis which is the smallest unit of the currency (Ether) in Ethereum held by the address.

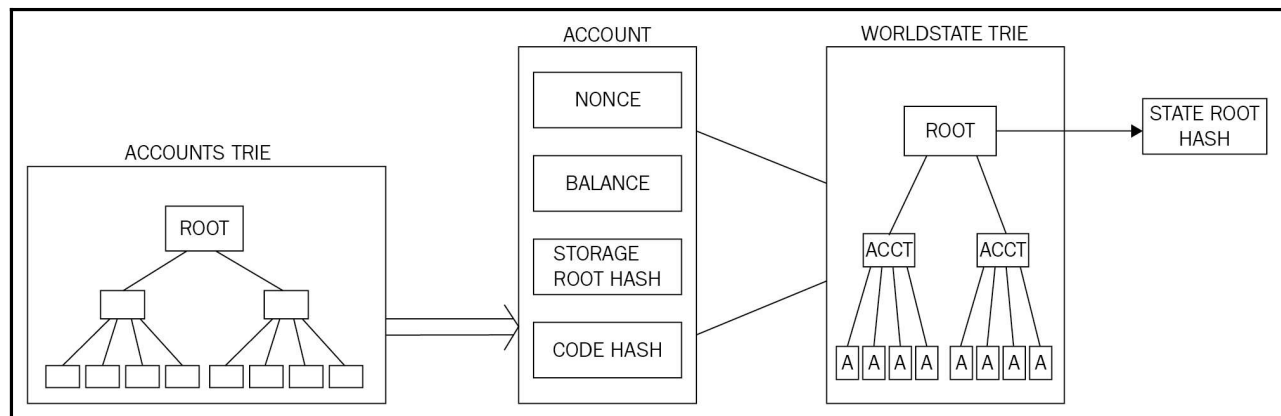
Storageroot

This field represents the root node of a Merkle Patricia tree that encodes the storage contents of the account.

Codehash

This is an immutable field that contains the hash of the smart contract code that is associated with the account. In the case of normal accounts, this field contains the Keccak 256-bit hash of an empty string. This code is invoked via a message call.

The world state and its relationship with accounts trie, accounts, and block header can be visualized in the following diagram. It shows the account data structure in the middle of the diagram, which contains a storage root hash derived from the root node of the account storage trie shown on the left. The account data structure is then used in the world state trie, which is a mapping between addresses and account states. Finally, the root node of the world state trie is hashed using the Keccak 256-bit algorithm and made part of the block header data structure, which is shown on the right-hand side of the diagram as state root hash.



Accounts trie (storage contents of account), account tuple, world state trie, and state root hash and their relationship

Accounts trie is basically a Merkle Patricia tree used to encode the storage contents of an account. The contents are stored as a mapping between keccak 256-bit hashes of 256-bit integer keys to the RLP-encoded 256-bit integer values.

Transactions

A transaction in Ethereum is a digitally signed data packet using a private key that contains the instructions that, when completed, either result in a message call or contract creation. Transactions can be divided into two types based on the output they produce:

- **Message call transactions:** This transaction simply produces a message call that is used to pass messages from one account to another.
- **Contract creation transactions:** As the name suggests, these transactions result in the creation of a new contract. This means that when this transaction is executed successfully, it creates an account with the associated code.

Both of these transactions are composed of a number of common fields, which are described here.

Nonce

Nonce is a number that is incremented by one every time a transaction is sent by the sender. It must be equal to the number of transactions sent and is used as a unique identifier for the transaction. A nonce value can only be used once.

gasPrice

The `gasPrice` field represents the amount of Wei required in order to execute the transaction.

gasLimit

The `gasLimit` field contains the value that represents the maximum amount of gas that can be consumed in order to execute the transaction. The concept of gas and gas limit will be covered later in the chapter in more detail. For now, it is sufficient to say that this is the amount of fee in Ether that a user (for example, the sender of the transaction) is willing to pay for computation.

To

As the name suggests, the `to` field is a value that represents the address of the recipient of the transaction.

Value

`Value` represents the total number of Wei to be transferred to the recipient; in the case of a contract account, this represents the balance that the contract will hold.

Signature

Signature is composed of three fields, namely v , r , and s . These values represent the digital signature (R , S) and some information that can be used to recover the public key (V). Also of the transaction from which the sender of the transaction can also be determined. The signature is based on ECDSA scheme and makes use of the SECP256k1 curve. The theory of elliptic curve cryptography was discussed in [Chapter 3, Cryptography and technical foundations](#). In this section, ECDSA will be presented in the context of its usage in Ethereum.

V is a single byte value that depicts the size and sign of the elliptic curve point and can be either 27 or 28. V is used in the ECDSA recovery contract as a recovery ID. This value is used to recover (derive) the public key from the private key. In secp256k1, the recovery ID is expected to be either 0 or 1. In Ethereum, this is offset by 27. More details on the ECDSARECOVER function will be provided later in this chapter.

R is derived from a calculated point on the curve. First, a random number is picked up, which is multiplied with the generator of the curve to calculate a point on the curve. The x coordinate part of this point is R . R is encoded as a 32 byte sequence. R must be greater than 0 and less than the secp256k1n limit (115792089237316195423570985008687907852837564279074904382605163141518161494337).

S is calculated by multiplying R with the private key and adding it into the hash of the message to be signed and by finally dividing it with the random number chosen to calculate R . S is also a 32 byte sequence. R and S together represent the signature.

In order to sign a transaction, the `ECDSASIGN` function is used, which takes the message to be signed and the private key as an input and produces V , a single byte value; R , a 32 byte value, and S , another 32 byte value. The equation is as follows:

$$ECDSASIGN(\text{Message}, \text{Private Key}) = (V, R, S)$$

Init

The `Init` field is used only in transactions that are intended to create contracts. This represents a byte array of unlimited length that specifies the EVM code to be used in the account initialization process. The code contained in this field is executed only once, when the account is created for the first time, and gets destroyed immediately after that.

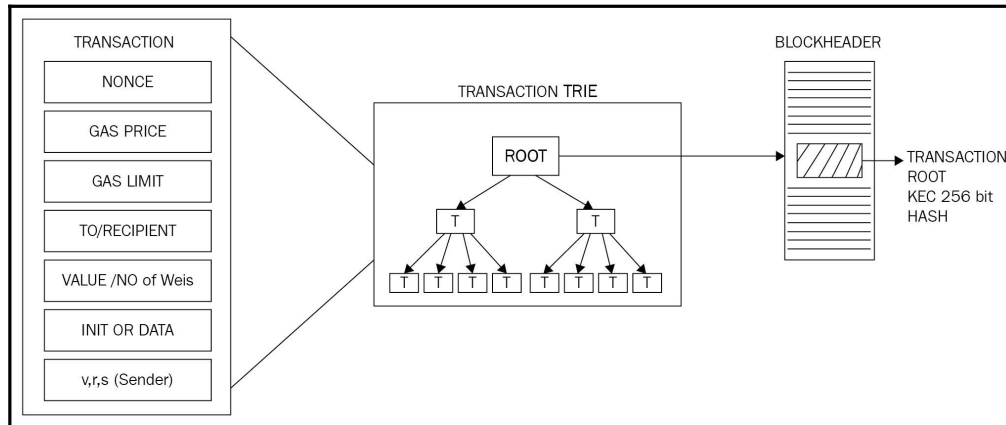
`Init` also returns another code section called *body*, which persists and runs in response to message calls that the contract account may receive. These message calls may be sent via a transaction or an internal code execution.

Data

If the transaction is a message call, then the `data` field is used instead of `init`, which represents the input data of the message call. It is also unlimited in size and is organized as a byte array.

This can be visualized in the following diagram, where a transaction is a tuple of the fields mentioned earlier, which is then included in a transaction trie (a modified Merkle-Patricia tree) composed of the transactions to be included. Finally, the root node of transaction trie is hashed using a Keccak 256-bit algorithm and is included in the block header along with a list of transactions in the block.

Transactions can be found in either transaction pools or blocks. When a mining node starts its operation of verifying blocks, it starts with the highest paying transactions in the transaction pool and executes them one by one. When the gas limit is reached or no more transactions are left to be processed in the transaction pool, the mining starts. In this process, the block is repeatedly hashed until a valid nonce is found that, once hashed with the block, results in a value less than the difficulty target. Once the block is successfully mined, it will be broadcasted immediately to the network, claiming success, and will be verified and accepted by the network. This process is similar to Bitcoin's mining process discussed in the previous chapter. The only difference is that Ethereum's Proof of Work algorithm is ASIC-resistant, known as *Ethash*, where finding a nonce requires large memory.



Relationship between transaction, transaction trie and block header

Contract creation transaction

There are a few essential parameters that are required when creating an account. These parameters are listed as follows:

- Sender
- Original transactor
- Available gas
- Gas price
- Endowment, which is the amount of ether allocated initially
- A byte array of arbitrary length
- Initialization EVM code
- Current depth of the message call/contract-creation stack (current depth means the number of items that are already there in the stack)

Addresses generated as a result of contract creation transaction are 160-bit in length. Precisely, as defined in the yellow paper, they are the rightmost 160-bits of the Keccak hash of the RLP encoding of the structure containing only the sender and the nonce. Initially, the nonce in the account is set to zero. The balance of the account is set to the value passed to the contract. Storage is also set to empty. Code hash is Keccak 256-bit hash of the empty string.

The account is initialized when the EVM code (Initialization EVM code) is executed. In the case of any exception during code execution, such as not having enough gas, the state does not change. If the execution is successful, then the account is created after the payment of appropriate gas costs. The current version of Ethereum (homestead) specifies that the result of contract transaction is either a new contract with its balance, or no new contract is created with no transfer of value. This is in contrast to previous versions, where the contract could be created regardless of the contract code deployment being successful or not due to an out-of-gas exception.

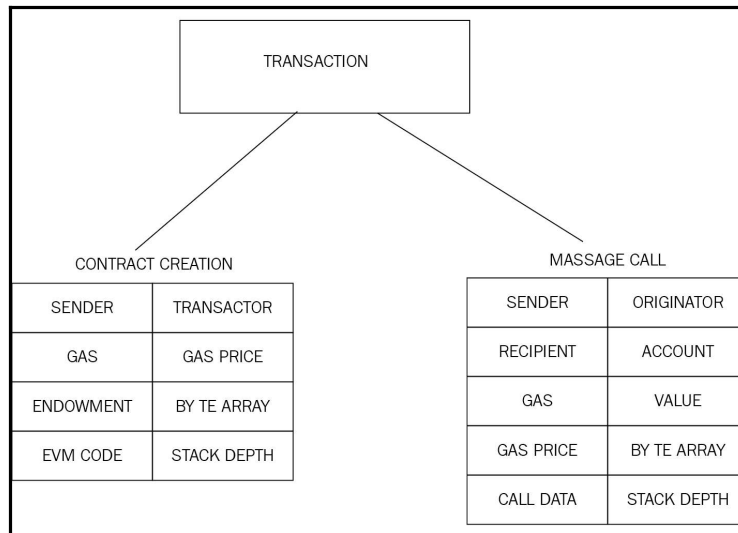
Message call transaction

A message call requires several parameters for execution, which are listed as follows:

- Sender
- The transaction originator
- Recipient
- The account whose code is to be executed
- Available gas
- Value
- Gas price
- Arbitrary length byte array
- Input data of the call
- Current depth of the message call/contract creation stack

Message calls result in state transition. Message calls also produce output data, which is not used if transactions are executed. In cases where message calls are triggered by VM code, the output produced by the transaction execution is used.

In the following diagram, the segregation between two types of transaction is shown:



Types of transactions, required parameters for execution

Elements of the Ethereum blockchain

In the following section, you will be introduced to various components of the Ethereum network and the blockchain. First, the basic concept of the EVM is given in the next section.

Ethereum virtual machine (EVM)

EVM is a simple stack-based execution machine that runs bytecode instructions in order to transform the system state from one state to another. The word size of the virtual machine is set to 256-bit. The stack size is limited to 1024 elements and is based on the **LIFO (Last in First Out)** queue. EVM is a Turing-complete machine but is limited by the amount of gas that is required to run any instruction. This means that infinite loops that can result in denial of service attacks are not possible due to gas requirements. EVM also supports exception handling in case exceptions occur, such as not having enough gas or invalid instructions, in which case the machine would immediately halt and return the error to the executing agent.

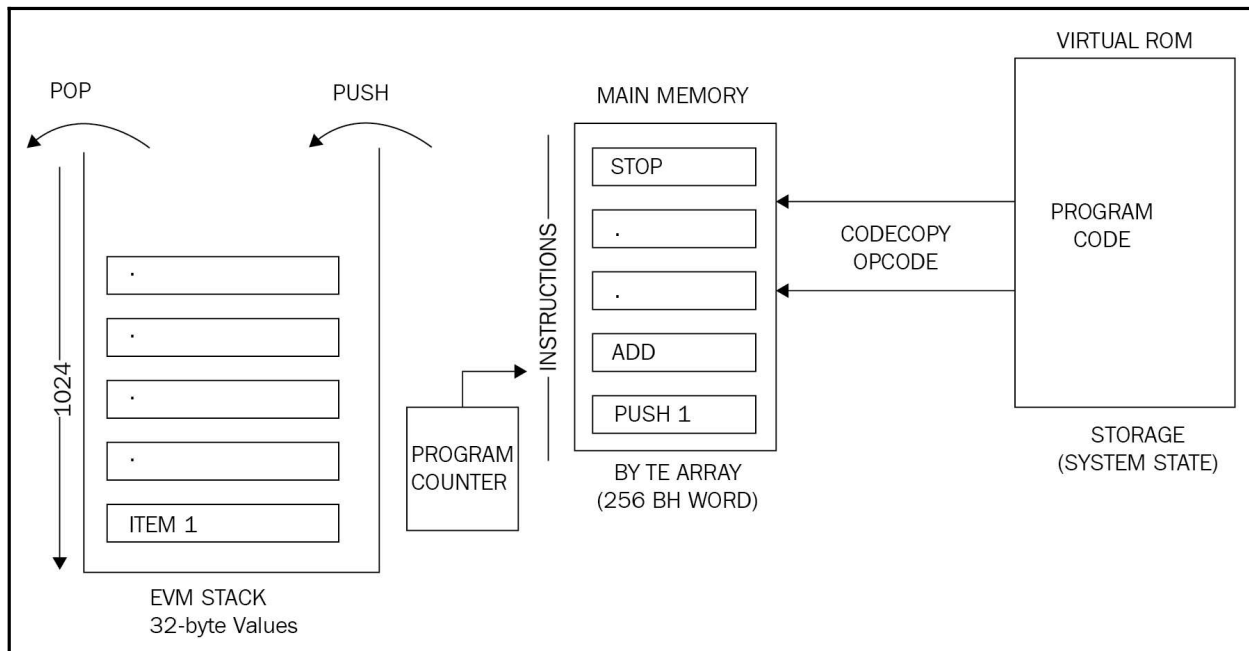
EVM is a fully isolated and sandboxed runtime environment. The code that runs on the EVM does not have access to any external resources, such as a network or filesystem.

As discussed earlier, EVM is a stack-based architecture. EVM is big-endian by design and it uses 256-bit wide words. This word size allows for Keccak 256-bit hash and elliptic curve cryptography computations.

There are two types of storage available to contracts and EVM. The first one is called memory, which is a byte array. When a contract finishes the code execution, the memory is cleared. It is akin to the concept of RAM. The other type, called storage, is permanently stored on the blockchain. It is a key value store.

Memory is unlimited but constrained by gas fee requirements. The storage associated with the virtual machine is a word addressable *word array* that is nonvolatile and is maintained as part of the system state. Keys and value are 32 bytes in size and storage. The program code is stored in a **virtual read-only memory (virtual ROM)** that is accessible using the CODECOPY instruction. The CODECOPY instruction is used to copy the program code into the main memory. Initially, all storage and memory is set to zero in the EVM.

The following diagram shows the design of the EVM where the virtual ROM stores the program code that is copied into main memory using **CODECOPY**. The main memory is then read by the EVM by referring to the program counter and executes instructions step by step. The program counter and EVM stack are updated accordingly with each instruction execution.



EVM operation

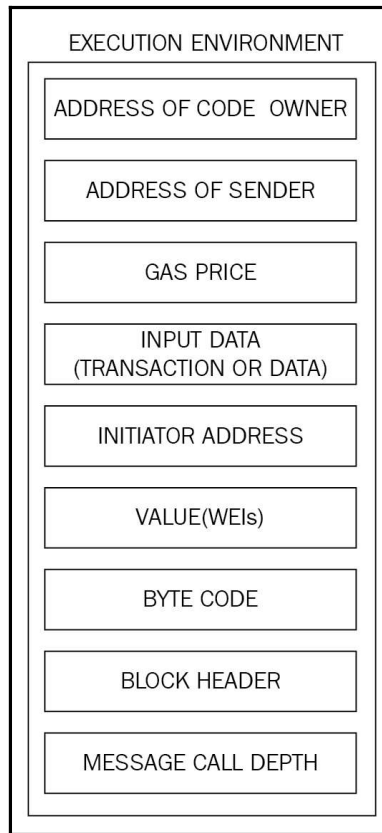
EVM optimization is an active area of research and recent research has suggested that EVM can be optimized and tuned to a very fine degree in order to achieve high performance. Research into the possibility of using **Web assembly (WASM)** is underway already. WASM is developed by Google, Mozilla, and Microsoft and is now being designed as an open standard by the W3C community group. The aim of WASM is to be able to run machine code in the browser that will result in execution at native speed. Similarly, the aim of EVM 2.0 is to be able to run the EVM instruction set (Opcodes) natively in CPUs, thus making it faster and efficient.

Execution environment

There are some key elements that are required by the execution environment in order to execute the code. The key parameters are provided by the execution agent, for example, a transaction. These are listed as follows:

1. The address of the account that owns the executing code.
2. The address of the sender of the transaction and the originating address of this execution.
3. The gas price in the transaction that initiated the execution.
4. Input data or transaction data depending on the type of executing agent. This is a byte array; in the case of a message call, if the execution agent is a transaction, then the transaction data is included as input data.
5. The address of the account that initiated the code execution or transaction sender. This is the address of the sender in case the code execution is initiated by a transaction; otherwise, it's the address of the account.
6. The value or transaction value. This is the amount in Wei. If the execution agent is a transaction, then it is the transaction value.
7. The code to be executed presented as a byte array that the iterator function picks up in each execution cycle.
8. The block header of the current block
9. The number of message calls or contract creation transactions currently in execution. In other words, this is the number of CALLs or CREATEs currently in execution.

The execution environment can be visualized as a tuple of nine elements, as follows:



Execution environment Tuple

In addition to the previously mentioned nine fields, system state and the remaining gas are also provided to the execution environment. The execution results in producing the resulting state, gas remaining after the execution, self-destruct or suicide set (described later), log series (described later), and any gas refunds.

Machine state

Machine state is also maintained internally by the EVM. Machine state is updated after each execution cycle of EVM. An iterator function (detailed in the next section) runs in the virtual machine, which outputs the results of a single cycle of the state machine. Machine state is a tuple that consist of the following elements:

- Available gas
- The program counter, which is a positive integer up to 256
- Memory contents

- Active number of words in memory
- Contents of the stack

The EVM is designed to handle exceptions and will halt (stop execution) in case any of the following exceptions occur:

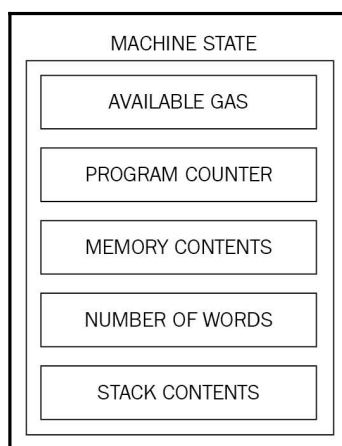
- Not having enough gas required for execution
- Invalid instructions
- Insufficient stack items
- Invalid destination of jump op codes
- Invalid stack size (greater than 1024)

The iterator function

The iterator function mentioned earlier performs various important functions that are used to set the next state of the machine and eventually the world state. These functions include the following:

- It fetches the next instruction from a byte array where the machine code is stored in the execution environment.
- It adds/removes (PUSH/POP) items from the stack accordingly.
- Gas is reduced according to the gas cost of the instructions/Opcodes.
- It increments the **program counter (PC)**.

Machine state can be viewed as a tuple shown in the following diagram:



Machine state tuple

The virtual machine is also able to halt in normal conditions if STOP or SUICIDE or RETURN Opcodes are encountered during the execution cycle.

Code written in a high-level language such as serpent, LLL, or Solidity is converted into the byte code that EVM understands in order for it to be executed by the EVM. Solidity is the high-level language that has been developed for Ethereum with JavaScript such as syntax to write code for smart contracts. Once the code is written, it is compiled into byte code that's understandable by the EVM using the Solidity compiler called solc.

LLL (Lisp-like Low-level language) is another language that is used to write smart contract code. Serpent is a Python-like high-level language that can be used to write smart contracts for Ethereum.

For example, a simple program in solidity is shown as follows:

```
pragma solidity ^0.4.0;
contract Test1
{
    uint x=2;
    function addition1(uint x) returns (uint y) {
        y=x+2;
    }
}
```

This program is converted into bytecode, as shown here. Details on how to compile solidity code with examples will be given in the next chapter.

Runtime byte code

```
606060405260e060020a6000350463989e17318114601c575b6000565b34600057602960043
5603b565b60408051918252519081900360200190f35b600281015b91905056
Opcodes PUSH1 0x60 PUSH1 0x40 MSTORE PUSH1 0x2 PUSH1 0x0 SSTORE CALLVALUE
PUSH1 0x0 JUMPI JUMPDEST PUSH1 0x45 DUP1 PUSH1 0x1A PUSH1 0x0 CODECOPY
PUSH1 0x0 RETURN PUSH1 0x60 PUSH1 0x40 MSTORE PUSH1 0xe0 PUSH1 0x2 EXP
PUSH1 0x0 CALLDATALOAD DIV PUSH4 0x989E1731 DUP2 EQ PUSH1 0x1C JUMPI
JUMPDEST PUSH1 0x0 JUMP JUMPDEST CALLVALUE PUSH1 0x0 JUMPI PUSH1 0x29 PUSH1
0x4 CALLDATALOAD PUSH1 0x3B JUMP JUMPDEST PUSH1 0x40 DUP1 MLOAD SWAP2 DUP3
MSTORE MLOAD SWAP1 DUP2 SWAP1 SUB PUSH1 0x20 ADD SWAP1 RETURN JUMPDEST
PUSH1 0x2 DUP2 ADD JUMPDEST SWAP2 SWAP1 POP JUMP
```

The identity function

The identity function is available at address 4 and is denoted by the ID. It simply defines output as input; in other words, whatever input is given to the ID function, it will output the same value. Gas requirement is calculated by a simple formula: $15 + 3 \lceil I_d/32 \rceil$ where I_d is the input data. This means that at a high level, the gas requirement is dependent on the size of the input data albeit with some calculation performed, as shown in the preceding equation.

All the previously mentioned precompiled contracts can become native extensions and can be included in the EVM opcodes in the future.

Accounts

Accounts are one of the main building blocks of the Ethereum blockchain. The state is created or updated as a result of the interaction between accounts. Operations performed between and on the accounts represent state transitions. State transition is achieved using what's called the Ethereum state transition function, which works as follows:

1. Confirm the transaction validity by checking the syntax, signature validity, and nonce.
2. Transaction fee is calculated and the sending address is resolved using the signature. Furthermore, sender's account balance is checked and subtracted accordingly and nonce is incremented. An error is returned if the account balance is not enough.
3. Provide enough ether (gas price) to cover the cost of the transaction. This is charged per byte incrementally according to the size of the transaction.
4. In this step, the actual transfer of value occurs. The flow is from the sender's account to receiver's account. The account is created automatically if the destination account specified in the transaction does not exist yet. Moreover, if the destination account is a contract, then the contract code is executed. This also depends on the amount of gas available. If enough gas is available, then the contract code will be executed fully; otherwise, it will run up to the point where it runs out of gas.
5. In cases of transaction failure due to insufficient account balance or gas, all state changes are rolled back with the exception of fee payment, which is paid to the miners.
6. Finally, the remainder (if any) of the fee is sent back to the sender as change and fee is paid to the miners accordingly. At this point, the function returns the resulting state.

Types of accounts

There are two types of accounts in Ethereum:

- Externally owned accounts
- Contract accounts

The first is **externally owned accounts (EOAs)** and the other is contract accounts. EOAs are similar to accounts that are controlled by a private key in bitcoin. Contract accounts are the accounts that have code associated with them along with the private key. An EOA has ether balance, is able to send transactions, and has no associated code, whereas a **Contract Account (CA)** has ether balance, associated code, and the ability to get triggered and execute code in response to a transaction or a message. It is worth noting that due to the Turing-completeness property of the Ethereum blockchain, the code within contract accounts can be of any level of complexity. The code is executed by EVM by each mining node on the Ethereum network. In addition, contract accounts are able to maintain their own permanent state and can call other contracts. It is envisaged that in the serenity release, the distinction between externally owned accounts and contract accounts may be eliminated.

Block

As discussed earlier, blocks are the main building blocks of a blockchain. Ethereum blocks consist of various components, which are described as follows:

- The block header
- The transactions list
- The list of headers of Ommers or Uncles

The transaction list is simply a list of all transactions included in the block. In addition, the list of headers of Uncles is also included in the block. The most important and complex part is the block header, which is discussed here.

Block header

Block headers are the most critical and detailed components of an Ethereum block. The header contains valuable information, which is described in detail here.

Parent hash

This is the Keccak 256-bit hash of the parent (previous) block's header.

Ommers hash

This is the Keccak 256-bit hash of the list of Ommers (Uncles) blocks included in the block.

Beneficiary

Beneficiary field contains the 160-bit address of the recipient that will receive the mining reward once the block is successfully mined.

State root

The state root field contains the Keccak 256-bit hash of the root node of the state trie. It is calculated after all transactions have been processed and finalized.

Transactions root

The transaction root is the Keccak 256-bit hash of the root node of the transaction trie. Transaction trie represents the list of transactions included in the block.

Receipts root

The receipts root is the keccak 256 bit hash of the root node of the transaction receipt trie. This trie is composed of receipts of all transactions included in the block. Transaction receipts are generated after each transaction is processed and contain useful post-transaction information. More details on transaction receipts are provided in the next section.

Logs bloom

The logs bloom is a bloom filter that is composed of the logger address and log topics from the log entry of each transaction receipt of the included transaction list in the block. Logging is explained in detail in the next section.

Difficulty

The difficulty level of the current block.

Number

The total number of all previous blocks; the genesis block is block zero.

Gas limit

The field contains the value that represents the limit set on the gas consumption per block.

Gas used

The field contains the total gas consumed by the transactions included in the block.

Timestamp

Timestamp is the epoch Unix time of the time of block initialization.

Extra data

Extra data field can be used to store arbitrary data related to the block.

Mixhash

Mixhash field contains a 256-bit hash that once combined with the nonce is used to prove that adequate computational effort has been spent in order to create this block.

Nonce

Nonce is a 64-bit hash (a number) that is used to prove, in combination with the mixhash field, that adequate computational effort has been spent in order to create this block.

State and nonce validation

Verify the state and nonce. In the case of mining, compute a valid state and nonce.

Block difficulty

Block difficulty is increased if the time between two blocks decreases, whereas it increases if the block time between two blocks decreases. This is required to maintain a roughly consistent block generation time. The difficulty adjustment algorithm in Ethereum's homestead release is shown as follows:

```
block_diff = parent_diff + parent_diff // 2048 *  
max(1 - (block_timestamp - parent_timestamp) // 10, -99) +  
int(2**((block.number // 100000) - 2))
```

The preceding algorithm means that, if the time difference between the generation of the parent block and the current block is less than 10 seconds, the difficulty goes up. If the time difference is between 10 to 19 seconds, the difficulty level remains the same. Finally, if the time difference is 20 seconds or more, the difficulty level decreases. This decrease is proportional to the time difference.

In addition to timestamp-difference-based difficulty adjustment, there is also another part (shown in the last line of the preceding algorithm) that increases the difficulty exponentially after every 100,000 blocks. This is the so called *difficulty time bomb* or *Ice age* introduced in the Ethereum network, which will make it very hard to mine on the Ethereum blockchain at some point in the future. This will encourage users to switch to Proof of Stake as mining on the POW chain will eventually become prohibitively difficult. According to the latest update and estimates based on the algorithm, the block generation time will become significantly high during the second half of the year 2017 and in 2021, it will become so high that it will be virtually impossible to mine on the POW chain. This way, miners will have no choice but to switch to the Proof of Stake scheme proposed by Ethereum called Casper.

Ether

Ether is minted by miners as a currency reward for the computational effort they spend in order to secure the network by verifying and with validation transactions and blocks. Ether is used within the Ethereum blockchain to pay for the execution of contracts on the EVM. Ether is used to purchase gas as crypto fuel, which is required in order to perform computation on the Ethereum blockchain.

The denomination table is shown as follows:

Unit	Wei Value	Weis
Wei	1 Wei	1
Babbage	1e3 Wei	1,000
Lovelace	1e6 Wei	1,000,000
Shannon	1e9 Wei	1,000,000,000
Szabo	1e12 Wei	1,000,000,000,000
Finney	1e15 Wei	1,000,000,000,000,000
Ether	1e18 Wei	1,000,000,000,000,000,000

Fees are charged for each computation performed by the EVM on the blockchain. A detailed fee schedule is shown in the upcoming section.

Gas

Gas is required to be paid for every operation performed on the ethereum blockchain. This is a mechanism that ensures that infinite loops cannot cause the whole blockchain to stall due to the Turing-complete nature of the EVM. A transaction fee is charged as some amount of Ether and is taken from the account balance of the transaction originator. A fee is paid for transactions to be included by miners for mining. If this fee is too low, the transaction may never be picked up; the more the fee, the higher are the chances that the transactions will be picked up by the miners for inclusion in the block. Conversely, if the transaction that has an appropriate fee paid is included in the block by miners but has too many complex operations to perform, it can result in an out-of-gas exception if the gas cost is not enough. In this case, the transaction will fail but will still be made part of the block and the transaction originator will not get any refund.

Transaction cost can be estimated using the following formula:

$$\text{Total cost} = \text{gasUsed} * \text{gasPrice}$$

Here, *gasUsed* is the total gas that is supposed to be used by the transaction during the execution and *gasPrice* is specified by the transaction originator as an incentive to the miners to include the transaction in the next block. This is specified in Ether. Each EVM opcode has a fee assigned to it. It is an estimate because the gas used can be more or less than the value specified by the transaction originator originally. For example, if computation takes too long or the behavior of the smart contract changes in response to some other factors, then the transaction execution may perform more or less operations than originally intended and can result in consuming more or fewer gas. If the execution runs out of gas, everything is immediately rolled back; otherwise, if the execution is successful and there is some remaining gas, then it is returned to the transaction originator.

Each operation costs some gas; a high level fee schedule of a few operations is shown as an example here:

Operation Name	Gas Cost
step	1
stop	0
suicide	0
sha3	30
sload	20
txdata	5
transaction	500
contract creation	53000

Based on the preceding fee schedule and the formula discussed earlier, an example calculation of the SHA3 operation can be calculated as follows:

- SHA3 costs 30 gas
- Current gas price is 25 GWei, which is 0.000000025 Ether
- Multiplying both: $0.000000025 * 30 = 0.00000075$ Ether

In total, 0.00000075 Ether is the total gas that will be charged.

Fee schedule

Gas is charged in three scenarios as a prerequisite to the execution of an operation:

- The computation of an operation
- For contract creation or message call
- Increase in the usage of memory

A list of instructions and various operations with the gas values has been provided previously in the chapter.

Messages

Messages, as defined in the yellow paper, are the data and value that are passed between two accounts. A message is a data packet passed between two accounts. This data packet contains data and value (amount of ether). It can either be sent via a smart contract (autonomous object) or from an external actor (externally owned account) in the form of a transaction that has been digitally signed by the sender.

Contracts can send messages to other contracts. Messages only exist in the execution environment and are never stored. Messages are similar to transactions; however, the main difference is that they are produced by the contracts, whereas transactions are produced by entities external (externally owned accounts) to the Ethereum environment.

A message consists of the components mentioned here:

1. Sender of the message
2. Recipient of the message
3. Amount of Wei to transfer and message to the contract address
4. Optional data field (Input data for the contract)
5. Maximum amount of gas that can be consumed

Messages are generated when `CALL` or `DELEGATECALL` Opcodes are executed by the contracts.

Calls

A call does not broadcast anything to the blockchain; instead, it is a local call to a contract function and runs locally on the node. It is almost like a local function call. It does not consume any gas as it is a read-only operation. It is akin to a dry run. Calls are executed locally on a node and generally do not result in any state change. As defined in the yellow paper, this is the act of passing a message from one account to another. If the destination account has an associated EVM code, then the virtual machine will start upon the receipt of the message to perform the required operations. If the message sender is an autonomous object, then the call passes any data returned from the virtual machine operation.

State is altered by transactions. These are created by external factors and are signed and then broadcasted to the Ethereum network.

Mining

Mining is the process by which new currency is added to the blockchain. This is an incentive for the miners to validate and verify blocks made up of transactions. The mining process helps secure the network by verifying computations.

At a theoretical level, a miner performs the following functions:

1. Listens for the transactions broadcasted on the Ethereum network and determines the transactions to be processed.
2. Determines stale blocks called Uncles or Ommers and includes them in the block.
3. Updates the account balance with the reward earned from successfully mining the block.
4. Finally, a valid state is computed and block is finalized, which defines the result of all state transitions.

The current method of mining is based on Proof of Work, which is similar to that of bitcoin. When a block is deemed valid, it has to satisfy not only the general consistency requirements, but it must also contain the Proof of Work for a given difficulty.

The Proof of Work algorithm is due to be replaced with the Proof of Stake algorithm with the release of serenity. Considerable research work has been carried out in order to build the Proof of Stake algorithm suitable for the Ethereum Network.

An Algorithm named Casper has been developed, which will replace the existing Proof of Work algorithm in Ethereum. This is a security deposit based on the economic protocol where nodes are required to place a security deposit before they can produce blocks. Nodes have been named bonded validators in Casper, whereas the act of placing the security deposit is named bonding.

Ethash

Ethash is the name of the Proof of Work algorithm used in Ethereum. Originally, this was proposed as the Dagger-Hashimoto algorithm, but much has changed since the first implementation and the PoW algorithm has now evolved into what's known as Ethash now. Similar to bitcoin, the core idea behind mining is to find a nonce that once hashed the result in a predetermined difficulty level. Initially, the difficulty was low when Ethereum was new and even CPU and single GPU mining was profitable to a certain extent, but that is no longer the case. Now either pooled mining is profitable, or large GPU mining farms are used for mining purposes.

Ethash is a memory-hard algorithm, which makes it difficult to be implemented on specialized hardware. As in bitcoin, ASICs have been developed, which have resulted in mining centralization over the years, but memory-hard Proof of Work algorithms are one way of thwarting this threat and Ethereum implements Ethash to discourage ASIC development for mining. This algorithm requires choosing subsets of a fixed resource called **DAG (Directed Acyclic Graph)** depending on the nonce and block headers. DAG is around 2 GB in size and changes every 30000 blocks. Mining can only start when DAG is completely generated the first time a mining node starts. The time between every 30000 blocks is around 5.2 days and is called epoch. This DAG is used as a seed by the Proof of Work algorithm called Ethash. According to current specifications, the epoch time is defined as 30,000 blocks.

The current reward scheme is 5 Ether for successfully finding a valid nonce. In addition to receiving 5 Ethers, the successful miner also receives the cost of the gas consumed within the block and an additional reward for including stale blocks (Uncles) in the block. A maximum of two Uncles are allowed per block and are rewarded 7/8 of the normal block reward. In order to achieve a 12 second block time, block difficulty is adjusted at every block. The rewards are directly proportional to the miner's hash rate, which basically means how fast a miner can hash.

Mining can be performed by simply joining the Ethereum network and running an appropriate client. The key requirement is that the node should be fully synced with the main network before mining can start.

In the upcoming section, various methods of mining are mentioned.

CPU mining

Even though not profitable on the main net, CPU mining is still valuable on the test network or even a private network to experiment with mining and contract deployment. Private and test networks will be discussed with practical examples in the next chapter. A geth example is shown on how to start CPU mining here. Geth can be started with mine switch in order to start mining:

```
geth --mine --minerthreads <n>
```

CPU mining can also be started using the web 3 geth console. Geth console can be started by issuing the following command:

```
geth attach
```

After this, the miner can be started by issuing the following command, which will return true if successful, or false otherwise. Take a look at the following command:

```
Miner.start(4)  
True
```

The preceding command will start the miner with four threads. Take a look at the following command:

```
Miner.stop  
True
```

The preceding command will stop the miner. The command will return true if successful.

GPU mining

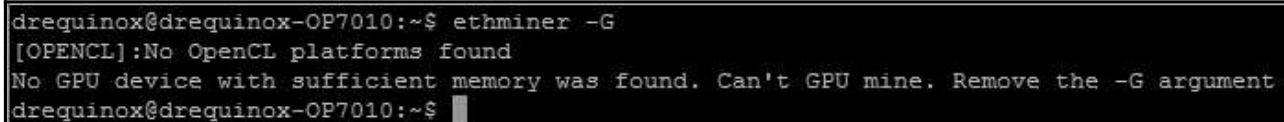
At a basic level, GPU mining can be performed easily by running two commands:

```
geth --rpc
```

Once geth is up and running and the blockchain is fully downloaded, Ethminer can be run in order to start mining. Ethminer is a standalone miner that can also be used in the farm mode to contribute to mining pools. It can be downloaded from <https://github.com/Genoil/cpp-ethereum/tree/master/releases>:

ethminer -G

Running with G switch assumes that the appropriate graphics card is installed and configured correctly. If no appropriate graphics cards are found, ethminer will return an error, as shown in the following screenshot:



```
drequinox@drequinox-OP7010:~$ ethminer -G
[OPENCL]:No OpenCL platforms found
No GPU device with sufficient memory was found. Can't GPU mine. Remove the -G argument
drequinox@drequinox-OP7010:~$
```

Error in case no appropriate GPUs can be found

GPU mining requires an AMD or Nvidia graphics card and an applicable OpenCL SDK. For NVidia chipset, it can be downloaded from <https://developer.nvidia.com/cuda-downloads>. For AMD chipsets, it is available at <http://developer.amd.com/tools-and-sdks/opencl-zone/amd-accelerated-parallel-processing-app-sdk>.

Once the graphics cards are installed and configured correctly, the process can be started by issuing the `ethminer -G` command.

Ethminer can also be used to run benchmarking, as shown in the following screenshot. There are two modes that can be invoked for benchmarking. It can either be CPU or GPU. The commands are shown here.

CPU benchmarking

\$ ethminer -M -C

GPU benchmarking

\$ ethminer -M -G

The following screenshot example is shown for CPU mining benchmarking:

```
drequinox@drequinox-OP7010:~$ ethminer -M -C
◇ 22:43:30.560 ethminer: #00004000...
Benchmarking on platform: 8-thread CPU
Preparing DAG...
□ 22:43:30.561 miner0 Loading full DAG of seedhash: #00000000...
Warming up...
Trial 1... 0
Trial 2... DAG 22:43:38.310 miner0 Generating DAG file. Progress: 0 %
0
Trial 3... 0
Trial 4... DAG 22:43:45.336 miner0 Generating DAG file. Progress: 1 %
0
```

CPU benchmarking

The GPU device to be used can also be specified in the command line:

```
$ ethminer -M -G --opencl-device 1
```

As GPU mining is implemented using OpenCL AMD, chipset-based GPUs tend to work faster as compared to NVidia GPUs. Due to the high memory requirements (DAG creation), FPGAs and ASICs will not provide any major advantage over GPUs. This is done on purpose in order to discourage the development of specialized hardware for mining.

Mining rigs

As difficulty increased over time for mining Ether, mining rigs with multiple GPUs were starting to be built by the miners. A mining rig usually contains around five GPU cards, and all of them work in parallel for mining, thus improving the chances of finding valid nonces for mining.

Mining rigs can be built with some effort and are also available commercially from various vendors. A typical mining rig configuration includes the components discussed in the upcoming sections.

Motherboard

A specialized motherboard with multiple PCI-E x1 or x16 slots, for example, BIOSTAR Hi-Fi or ASRock H81, is required.

SSD hard drive

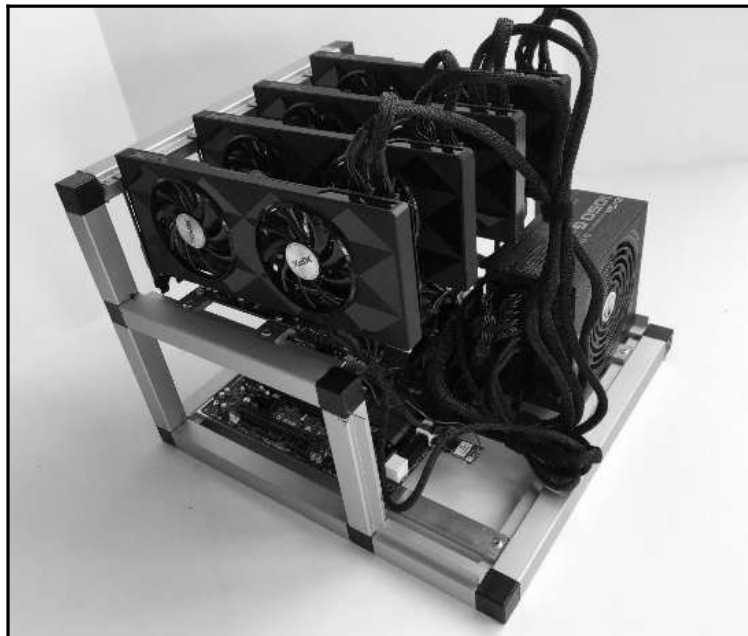
An SSD hard drive is required. The SSD drive is recommended because of its much faster performance over the analog equivalent. This will be mainly used to store the blockchain.

GPU

The GPU is the most important component of the rig as it is the main workhorse that will be used for mining. For example, it can be a Sapphire AMD Radeon R9 380 with 4 GB RAM.

Linux Ubuntu's latest version is usually chosen as the operating system for the rig. There is also another variant of Linux available, called EthOS (available at <http://ethosdistro.com/>), that is especially built for Ethereum mining and supports mining operations natively.

Finally, mining software such as Ethminer and geth are installed. Additionally, some remote monitoring and administration software is also installed so that rigs can be monitored and managed remotely, if required. It is also important to put appropriate air conditioning or cooling mechanisms in place as running multiple GPUs can generate a lot of heat. This also necessitates the need for using an appropriate monitoring software that can alert users if there are any problems with the hardware, for example, if the GPUs are overheating.



A mining rig for Ethereum for sale at eBay

Mining pools

There are many online mining pools that offer Ethereum mining. Ethminer can be used to connect to a mining pool using the following command. Each pool publishes its own instructions, but generally, the process of connecting to a pool is similar. An example from `ethereumpool.co` is shown here:

```
ethminer -C -F
http://ethereumpool.co/?miner=0.1@0x024a20cc5feba7f3dc3776075b3e60c20eb1459
c@DrEquinox
```



```
drequinox@drequinox-OP7010:~$ ethminer -C -F http://ethereumpool.co/?miner=0.1@0x024a20cc5feba7f3dc3776075b3e60c20eb1459c@DrEquinox
miner 23:50:52.046 ethminer Getting work package...
```

Screenshot of ethminer

Clients and wallets

As Ethereum is under heavy development and evolution, there are many components, clients, and tools that have been developed and introduced over the last few years. The following is a list of all main components, client software, and tools that are available with Ethereum. This list is provided in order to reduce the ambiguity around many tools and clients available for Ethereum. The list provided here also explains the usage and significance of various components.

Geth

This is the Go implementation of the Ethereum client.

Eth

This is the C++ implementation of the Ethereum client.

Pyethapp

This is the Python implementation of the Ethereum client.